

Pipeline Parallelism - Microbatching

Basic Pipeline Problem:

Time →

Device 1:	[Compute]	[Idle]	[Idle]	[Idle]
Device 2:	[Idle]	[Compute]	[Idle]	[Idle]
Device 3:	[Idle]	[Idle]	[Compute]	[Idle]
Device 4:	[Idle]	[Idle]	[Idle]	[Compute]

This naive "layer-wise" parallelization is inefficient because it's sequential. Only one device works at a time while the others sit idle, leading to poor hardware utilization.

Pipeline Solution with Microbatches:

Time →

Device 1:	[F ₀]	[F ₁]	[F ₂]	[F ₃]	[B ₃]	[B ₂]	[B ₁]	[B ₀]
Device 2:		[F ₀]	[F ₁]	[F ₂]	[B ₂]	[B ₁]	[B ₀]	
Device 3:			[F ₀]	[F ₁]	[B ₁]	[B ₀]		
Device 4:				[F ₀]	[B ₀]			

Splitting the input into microbatches allows devices to work simultaneously on different data chunks. This overlaps computation and drastically cuts idle time ("bubbles"), reducing them from $(m-1)$ to $(m-1)/b$ (where m is stages, b is microbatches).

F = Forward pass, B = Backward pass, subscript = microbatch number

Model Parallelism Strategies

When to use

- Model doesn't fit on one device (memory)
- Boost throughput when data parallelism alone saturates

Layer/pipeline parallelism

- Partition consecutive layers into K stages placed on different devices
- Forward: activations flow stage 1 $\rightarrow$... $\rightarrow$ K
- Backward: gradients flow K $\rightarrow$... $\rightarrow$ 1

Keep the pipeline full

- Split each global batch into M microbatches
- While stage s handles microbatch i, stage s+1 handles i-1, etc.
- Aim: minimize “bubbles” (idle time) at pipeline start/end

Practical goals and knobs

- Balance stage runtimes (group light layers together; profile and iterate)
- Manage activation memory: activation checkpointing trades extra compute for lower memory
- Choose M (microbatch count) to trade utilization vs. latency/memory
- Overlap inter-stage communication with compute when possible
- Combine with data parallelism for scale (hybrid parallelism)



Pipeline Parallelism: Utilization vs. Memory

Let:

- K = number of pipeline stages
- M = number of microbatches

For balanced stages, approximate pipeline utilization is:

$$Utilization \approx \frac{M}{M + K - 1}$$

So, the bubble fraction is:

$$Bubble\ fraction \approx \frac{K - 1}{M + K - 1}$$

When $M \gg K$:

$$Bubble\ fraction \approx \frac{K - 1}{M}$$

Schedulers

- GPipe: all forward passes, then all backward passes
 - simple and consistent;
 - high activation-memory cost.
- 1F1B: interleaves one forward and one backward pass
 - lower activation-memory cost;
 - better utilization after warm-up.

Sync vs Async vs Elastic Data Parallel

Synchronous all-reduce:

- Pros: stable convergence; single-node equivalence
- Cons: straggler-sensitive; per-step barrier

Parameter-server (async/stale-sync):

- Pros: more throughput; tolerant to slow clients
- Cons: stale gradients; careful LR tuning; potential accuracy hit

Elastic DP (synchronous with changing world size):

- World size adjusts at runtime; reshard data/checkpoints; adjust LR with effective batch

Guidance: prefer sync for accuracy-critical; use async/elastic when churn is inevitable

Topology-Aware & Hierarchical All-Reduce; Libraries

- Libraries:
 - NCCL (GPU-centric), Gloo (CPU/GPU), MPI (HPC)
- Algorithms:
 - Ring: bandwidth-optimal for large messages
 - Tree: lower latency for smaller messages
 - Reduce-scatter + all-gather: core of sharded training
- Hierarchical all-reduce:
 - Intra-node reduce-scatter (NVLink)
 - Inter-node reduce-scatter (IB/Ethernet)
 - Inter-node all-gather
 - Intra-node all-gather
- Practice: bucket sizes, algorithm selection per tensor size, topology hints